

Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

Факультет Программной Инженерии и Компьютерной Техники

Лабораторная работа №1

По дисциплине «**Бизнес-логика программных систем**»

Вариант №3001

Выполнили:

Студенты группы Р3306,

Михайлов Дмитрий Андреевич

Медведев Владислав Александрович

Преподаватель:

Кривоносов Егор Дмитриевич



Санкт-Петербург

2026 год

Содержание

Задание	3
Модель потока управления для автоматизируемого бизнес-процесса	3
UML-диаграммы классов и пакетов разработанного приложения	4
Спецификация REST API для всех публичных интерфейсов разработанного приложения	4
Исходный код системы или ссылка на репозиторий с исходным кодом	7
Вывод по работе	8

Задание

Вариант №3001: Skillbox — образовательная платформа с онлайн-курсами — <https://skillbox.ru/>. Бизнес-процесс: работа с каталогом, запись на курс (с оплатой).

Описать бизнес-процесс в соответствии с нотацией BPMN 2.0, после чего реализовать его в виде приложения на базе Spring Boot.

Порядок выполнения работы:

1. Выбрать один из бизнес-процессов, реализуемых сайтом из варианта задания.
2. Утвердить выбранный бизнес-процесс у преподавателя.
3. Специфицировать модель реализуемого бизнес-процесса в соответствии с требованиями BPMN 2.0.
4. Разработать приложение на базе Spring Boot, реализующее описанный на предыдущем шаге бизнес-процесс. Приложение должно использовать СУБД PostgreSQL для хранения данных, для всех публичных интерфейсов должны быть разработаны REST API.
5. Разработать набор curl-скриптов, либо набор запросов для REST клиента Insomnia для тестирования публичных интерфейсов разработанного программного модуля. Запросы Insomnia оформить в виде файла экспорта.
6. Развернуть разработанное приложение на сервере helios.

Модель потока управления для автоматизируемого бизнес-процесса

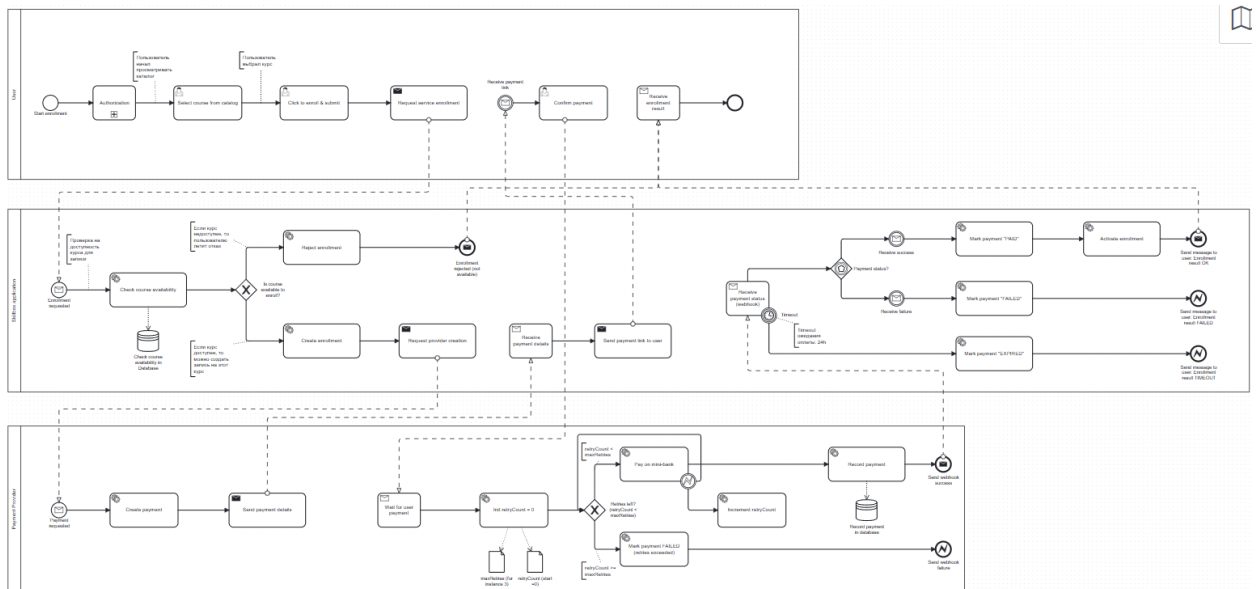


Рис. 1. BPMN диаграмма бизнес-процесса - запись на курс (с оплатой).

The UML Class Diagram, titled "Диаграмма классов", illustrates the architecture of the application. It is organized into several packages:

- dto**: Contains `CourseDTO` and `EnrollmentDTO`.
- controllers**: Contains `EnrollmentController`, `PaymentController`, and `CourseController`.
- services**: Contains `EnrollmentServiceImpl`, `PaymentServiceImpl`, `PaymentCallbackServiceImpl`, and `CourseServiceImpl`.
- repositories**: Contains `UserRepository`, `EnrollmentRepository`, `PaymentRepository`, and `CourseRepository`.
- entities**: Contains `Payment`, `Enrollment`, `PaymentStatus`, `User`, `EnrollmentStatus`, and `Course`.

Key relationships include:

- EnrollmentController** depends on `EnrollmentServiceImpl` and `EnrollmentRepository`.
- PaymentController** depends on `PaymentServiceImpl` and `PaymentRepository`.
- CourseController** depends on `CourseServiceImpl` and `CourseRepository`.
- EnrollmentServiceImpl** depends on `EnrollmentRepository` and `PaymentRepository`.
- PaymentServiceImpl** depends on `PaymentRepository` and `PaymentCallbackServiceImpl`.
- PaymentCallbackServiceImpl** depends on `PaymentRepository` and `PaymentStatus`.
- CourseServiceImpl** depends on `CourseRepository` and `EnrollmentRepository`.
- EnrollmentRepository** depends on `Enrollment` and `EnrollmentStatus`.
- PaymentRepository** depends on `Payment` and `PaymentStatus`.
- CourseRepository** depends on `Course` and `Enrollment`.
- Enrollment depends on `Payment` and `PaymentStatus`.**
- PaymentStatus depends on `Payment` and `EnrollmentStatus`.**
- User depends on `EnrollmentStatus`.**
- EnrollmentStatus depends on `Enrollment` and `Course`.**
- Course depends on `Enrollment`.**

Спецификация REST API для всех публичных интерфейсов разработанного приложения

1. Сценарий успешной записи на курс с оплатой

```
GET http://localhost:8080/api/courses/available
```

Затем создаётся запись пользователя на курс:

POST http://localhost:8080/api/enrollments

Тело запроса:

```
{
  "userId": 1,
  "courseId": 1
}
```

В ответе возвращается объект записи на курс. На этом этапе запись получает статус `PENDING_PAYMENT`, то есть ожидает оплату. Такой статус предусмотрен в перечислении `EnrollmentStatus`.

После этого создаётся платёж для данной записи:

POST <http://localhost:8080/api/payments/enrollment/{enrollmentId}>

В ответе возвращается объект платежа, содержащий:

- идентификатор платежа;
- внешний идентификатор `providerPaymentId`;
- ссылку на оплату `paymentUrl`;
- статус платежа `CREATED`.

Статус CREATED предусмотрен в перечислении PaymentStatus.

Далее имитируется ответ от банковского сервиса через webhook:
POST http://localhost:8080/api/payments/webhook

Тело запроса:

```
{
  "providerPaymentId": "provider-payment-id-from-payment-response",
  "status": "PAID"
}
```

После обработки webhook:

- статус платежа изменяется на PAID;
- статус записи на курс изменяется на ACTIVE;
- у курса уменьшается количество свободных мест на 1.

Финальная проверка выполняется запросами:

- GET http://localhost:8080/api/payments/enrollment/{enrollmentId}
- GET http://localhost:8080/api/enrollments/{enrollmentId}
- GET http://localhost:8080/api/courses/{courseId}

Ожидаемый результат:

- у платежа статус PAID;
- у записи статус ACTIVE;
- у курса количество доступных мест уменьшено.

2. Сценарий неуспешной оплаты

Сначала повторяются шаги создания записи на курс и создания платежа:

- POST http://localhost:8080/api/enrollments
- POST http://localhost:8080/api/payments/enrollment/{enrollmentId}

После этого имитируется неуспешный ответ от банковского сервиса:
POST http://localhost:8080/api/payments/webhook

Тело запроса:

```
{
  "providerPaymentId": "provider-payment-id-from-payment-response",
  "status": "FAILED",
  "failureReason": "Insufficient funds"
}
```

После обработки webhook:

- статус платежа изменяется на FAILED;

- статус записи на курс изменяется на PAYMENT_FAILED;
- в запись и в платёж сохраняется причина ошибки.

Проверка выполняется запросами:

- GET http://localhost:8080/api/payments/enrollment/{enrollmentId}
- GET http://localhost:8080/api/enrollments/{enrollmentId}

Ожидаемый результат:

- у платежа статус FAILED;
- у записи статус PAYMENT_FAILED;
- поле причины отказа заполнено.

Статусы FAILED и PAYMENT_FAILED предусмотрены в перечислениях PaymentStatus и EnrollmentStatus.

3. Сценарий истечения времени оплаты

При создании записи на курс ей назначается время истечения оплаты paymentExpiresAt. Для автоматической проверки просроченных оплат в приложении реализован планировщик PaymentExpirationScheduler, который запускается раз в 60 секунд.

Сначала создаются:

- запись на курс;
- платёж для этой записи.

После этого webhook не отправляется, то есть пользователь не завершает оплату.

Когда истекает время оплаты и срабатывает scheduler, приложение автоматически:

- переводит запись на курс из состояния PENDING_PAYMENT в PAYMENT_EXPIRED;
- переводит платёж в состояние EXPIRED.

Для проверки используются запросы:

- GET http://localhost:8080/api/enrollments/{enrollmentId}
- GET http://localhost:8080/api/payments/enrollment/{enrollmentId}

Ожидаемый результат:

- у записи статус PAYMENT_EXPIRED;
- у платежа статус EXPIRED.

Статусы PAYMENT_EXPIRED и EXPIRED также предусмотрены в перечислениях приложения.

4. Сценарий получения списка курсов

Для просмотра каталога курсов используется запрос:

GET <http://localhost:8080/api/courses>

Для просмотра только тех курсов, на которые ещё можно записаться:

GET <http://localhost:8080/api/courses/available>

В ответ возвращается список объектов курса, содержащих:

- идентификатор курса;
- название;
- описание;
- стоимость;
- валюту;
- вместимость;
- количество свободных мест;
- признак активности курса. Формат выходного DTO соответствует CourseDTO.

5. Обработка ошибок

В приложении реализован глобальный обработчик исключений `GlobalExceptionHandler`, который возвращает корректные HTTP-ответы при ошибках. Например:

- 404 Not Found — если курс, запись или платёж не найдены;
- 400 Bad Request — при бизнес-ошибках или некорректных входных данных;
- 500 Internal Server Error — при непредвиденной внутренней ошибке.

Например, если отправить запрос на создание записи без `userId` или `courseId`, приложение вернёт ошибку валидации, так как в запросе `CreateEnrollmentRequest` оба поля обязательны.

Исходный код системы или ссылка на репозиторий с исходным кодом

[Ссылка на репозиторий с кодом основного приложения.](#)

[Ссылка на репозиторий с кодом банковского приложения.](#)

Вывод по работе

В ходе выполнения лабораторной работы был рассмотрен и формализован бизнес-процесс образовательной платформы, связанный с работой с каталогом курсов, записью пользователя на курс и оплатой обучения. Для выбранного процесса была построена BPMN-модель, отражающая основные этапы обработки заявки: просмотр доступных курсов, создание записи на курс, формирование платежа, обработку результата оплаты и автоматическую обработку случая истечения времени оплаты.

На основе построенной модели было разработано приложение на базе Spring Boot с использованием СУБД PostgreSQL. В приложении реализованы сущности предметной области, репозитории, сервисный слой, REST API для всех публичных интерфейсов, а также обработка ошибок и планировщик для автоматического перевода просроченных оплат в соответствующее состояние. Кроме того, была предусмотрена интеграция с внешним банковским приложением через механизм создания платежа и webhook-уведомлений о результате оплаты.

В процессе работы были спроектированы UML-диаграммы классов и пакетов, отражающие структуру разработанного приложения, а также подготовлены и проверены сценарии тестирования через Postman. Проведённое тестирование показало корректную работу основных пользовательских сценариев: успешной оплаты, неуспешной оплаты, истечения времени оплаты и получения списка доступных курсов.

Таким образом, цель лабораторной работы была достигнута: бизнес-процесс был не только описан в нотации BPMN 2.0, но и реализован в виде работающего серверного приложения с REST API и хранением данных в PostgreSQL. В результате были получены практические навыки проектирования бизнес-процессов, построения серверных приложений на Spring Boot, организации взаимодействия между компонентами системы и тестирования REST-интерфейсов.